IlluminateAI SDK Integration

Roy Lovejoy
roy.lovejoy@illuminateAI.io
Draft: Feb 1, 2026

Changes since last release

- *Refactoring to IlluminateAI*
- *history() and pastMatch() functions (described in separate document)*

Transition Notes

Please note the transition from **RingoAI** modules to **IlluminateAI** modules.

Along with the new SPM url https://github.com/IlluminateAI-io/IlluminateAI-iOS-SDK , it is also required to replace any and all `import RingoAI` with `import IlluminateAI`

Introduction

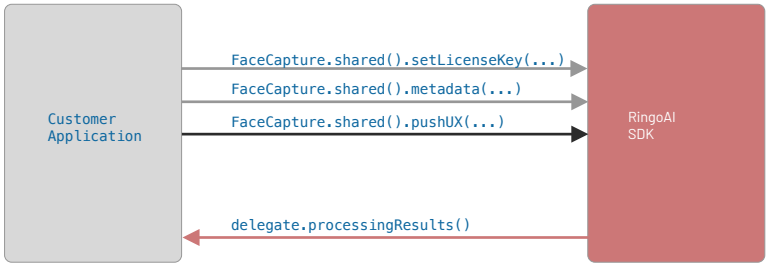
The IlluminateAI color matching SDK was designed to mimic standard iOS Frameworks, with simplicity of presenting the UX to have the action take place, and getting the resulting data back.

FaceCapture process overview

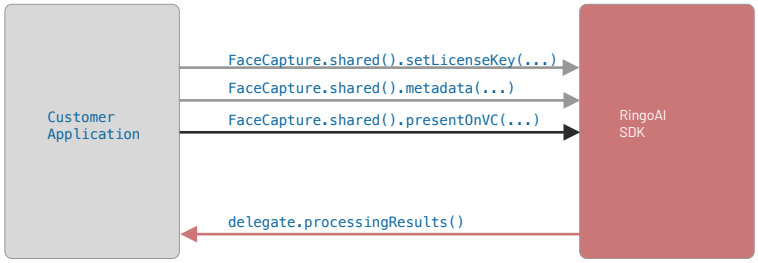
The IlluminateAI SDK provides a mechanism for presenting a “selfie camera” style user interface, guiding the user through calibration steps visually, then taking multiple photos under multiple lighting conditions, and calculating resulting product matches based on the color values obtained.

As with various iOS SDKs, user interface control is given to the IlluminateAI SDK, until the user cancels the process, or sees the process through to completion. All errors are handled internally. The two options for UX presentation are either a “push” onto the application’s **UINavigationController**, or a “present” on the applications hosting **UIViewController,** and once the user controlled process is completed, two delegate callbacks are called- first one is a process status, and the second one is a process results, that contains matching information.

Either:



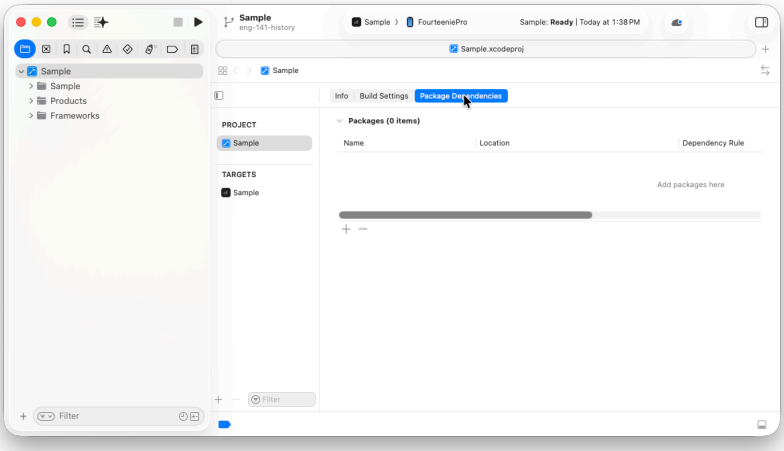
Or



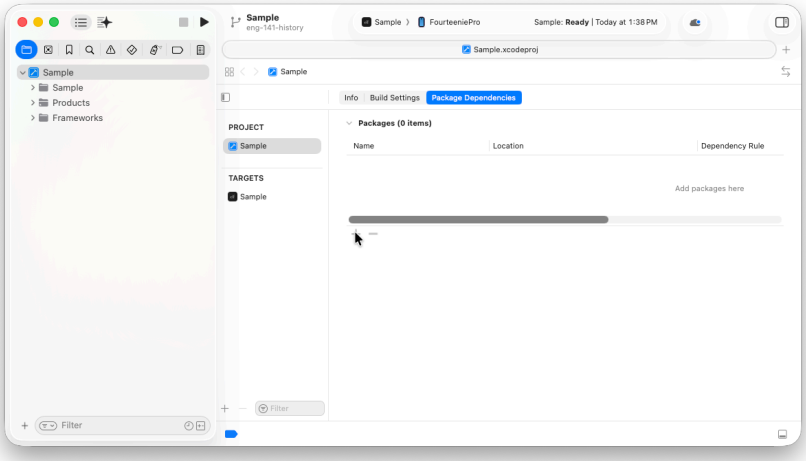
Configuring Xcode project

The IlluminateAI SDK uses the SPM delivery system, but for the immediate future, the ***MediaPipeTaskVision.xcframework*** dependency is still necessary**.**

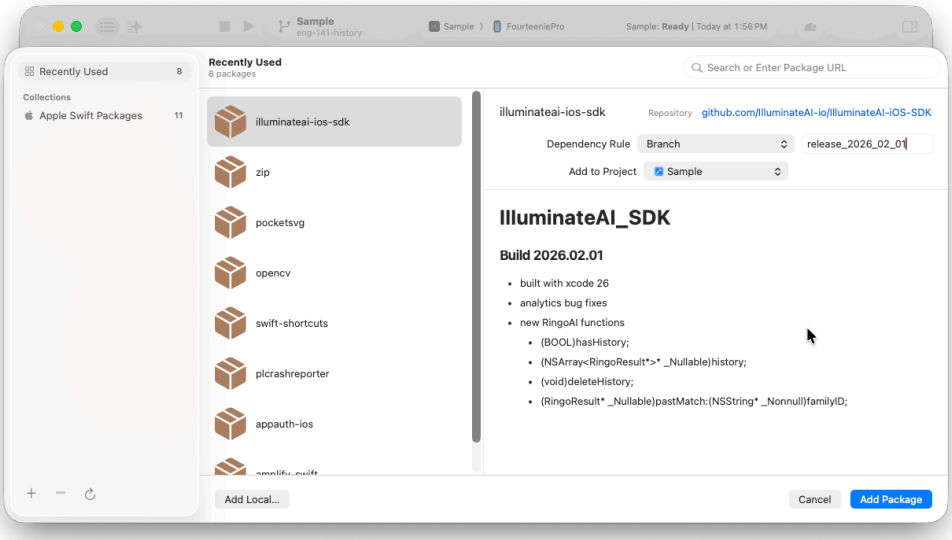
To add the IlluminateAI SDK SPM to your project, navigate to your app's **PROJECT** pane, **Package Dependencies** tab.



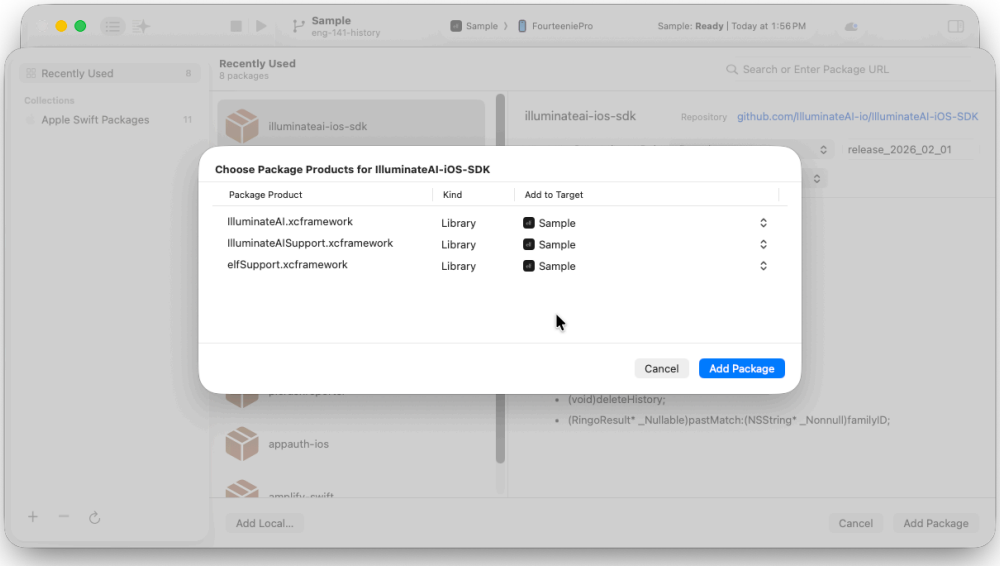
Tap on the + in the **Packages** section.



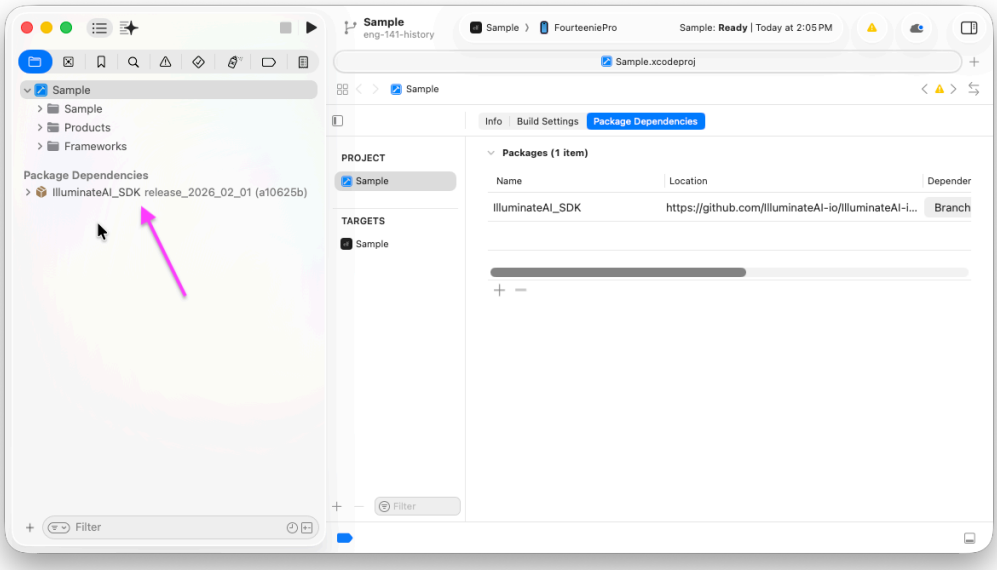
Set the SPM repository to https://github.com/IlluminateAI-io/IlluminateAI-iOS-SDK , set the branch to **release_2026_02_01** (or the current release desired) and tap “Add Package” button.



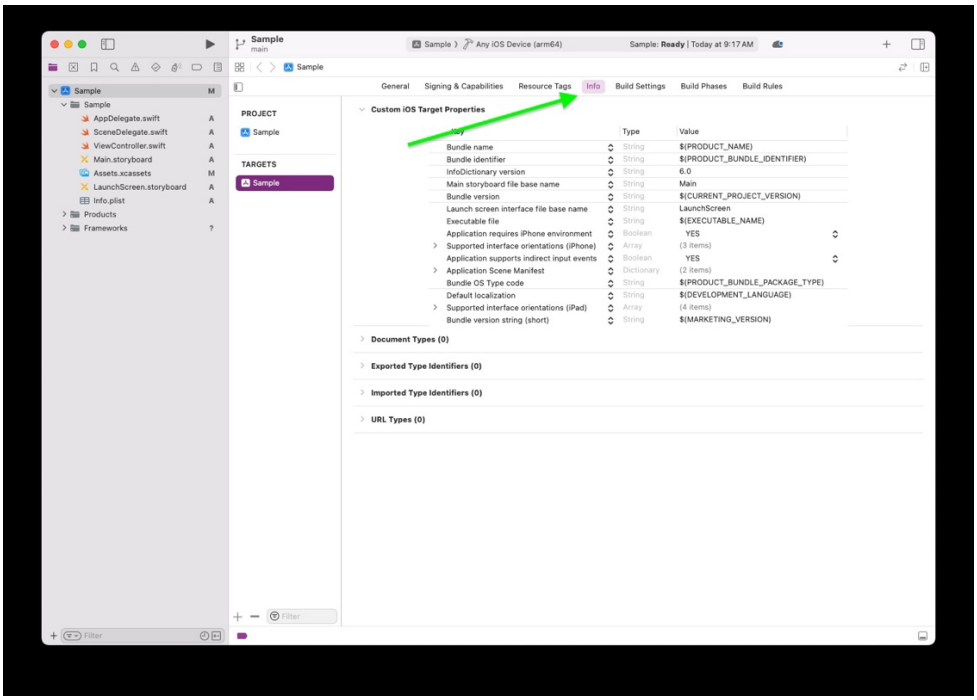
Then tap “Add Package” to add the three xcframeworks to your project.



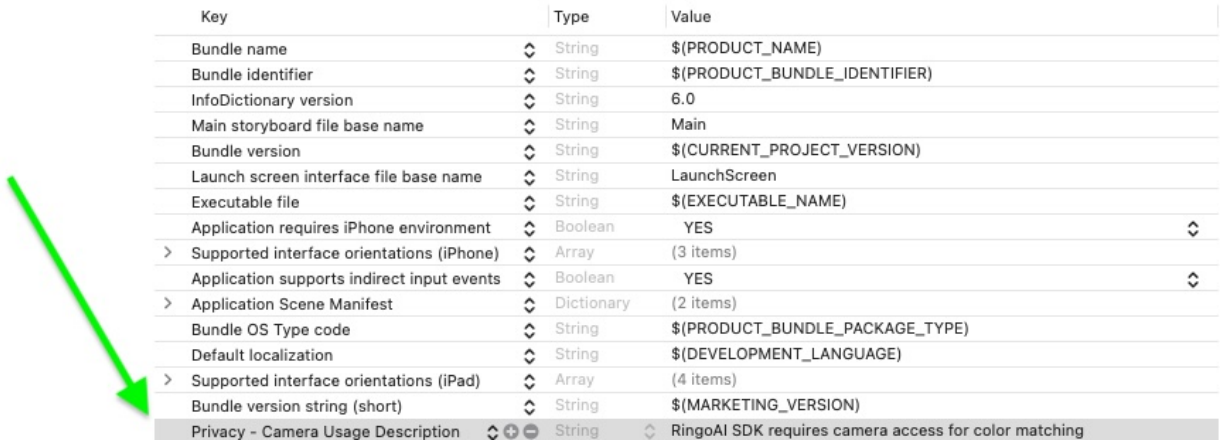
The IlluminateAI_SDK package dependencies should appear at the bottom left of your project window



Since the IlluminateAI.framework uses the iOS camera, permission is required to enable this. If permission in the app has not already been given, add this key/value pair to the **CUSTOM IOS TARGET PROPERTIES**.



Custom iOS Target Properties



Key	Type	Value
Bundle name	String	$(PRODUCT_NAME)
Bundle identifier	String	$(PRODUCT_BUNDLE_IDENTIFIER)
InfoDictionary version	String	6.0
Main storyboard file base name	String	Main
Bundle version	String	$(CURRENT_PROJECT_VERSION)
Launch screen interface file base name	String	LaunchScreen
Executable file	String	$(EXECUTABLE_NAME)
Application requires iPhone environment	Boolean	YES
Supported interface orientations (iPhone)	Array	(3 items)
Application supports indirect input events	Boolean	YES
Application Scene Manifest	Dictionary	(2 items)
Bundle OS Type code	String	$(PRODUCT_BUNDLE_PACKAGE_TYPE)
Default localization	String	$(DEVELOPMENT_LANGUAGE)
Supported interface orientations (iPad)	Array	(4 items)
Bundle version string (short)	String	$(MARKETING_VERSION)
Privacy - Camera Usage Description	String	RingoAI SDK requires camera access for color matching

Source Integration

The primary framework header that deals with integration is IlluminateAI /FaceCapture.h.

```
@protocol FaceCaptureDelegate <NSObject>

@optional

- (void)processingResults:(BOOL)success result:(RingoResult* _Nonnull)result;
- (void)thermalWarningGiven:(BOOL)abort;

@end

@interface FaceCapture : NSObject
@property (nonatomic, weak) id<FaceCaptureDelegate> delegate;

+ (FaceCapture* _Nonnull)shared;

- (NSError* _Nullable)setLicenseKey:(NSString* _Nonnull)key;
- (void)setUserID:(NSString* _Nonnull)userID;

- (void)pushUX:(UINavigationController* _Nonnull)navVC;
- (void)presentOnVC:(UIViewController* _Nonnull)vc;

- (void)popToFaceCapture;
- (void)abort;      // pops back to customer app

- (void)resultsExited;
- (void)cancelProcessingVC:(BOOL)animated;

@end
```

To make calls into the **ILLUMINATEAI.FRAMEWORK**, you will need to add the proper import statement to your code.

Swift:
```
import IlluminateAI
```
Objective C:
```
#import <IlluminateAI/IlluminateAI.h>
```

The `FaceCapture.metadata` call is used to pass the poqUserID and the productID into the SDK.

Swift:
```
FaceCapture.shared().metadata([userIDKey: kPoqUserID,
                               productIDKey: kProductID])
```

Objective C:
```
[FaceCapture.shared metadata:@{ RingoAI_userIDKey : kPoqUserID,
                                RingoAI_productIDKey : kProductID }];
```

There are two methods to activate the Illuminate capture mechanism, one of them is correct depending on your application's needs.

If you are using **UINAVIGATIONCONTROLLER** in your app structure, you may use

Swift:
```
FaceCapture.shared().pushUX(navigationController)
```
Objective C:
```
[FaceCapture.shared pushUX:self.navigationController];
```

To activate the FaceCapture sequence.

If you are not, you can use
Swift: `FaceCapture.shared().present(onVC: self)`
Objective C: `[FaceCapture.shared presentOnVC:self];`

And the UX will present on the current VC (self).

IlluminateAI SDK requires an initialization call in AppDelegate's DIDFINISHLAUNCHINGWITHOPTIONS, and a license key string to activate. **[future]**

Swift:

```
func application(_ application: UIApplication,
                 didFinishLaunchingWithOptions launchOptions:
                 [UIApplication.LaunchOptionsKey: Any]?) -> Bool {

    if let error = FaceCapture.shared().setLicenseKey("1234-5678-9ABC-DEFG") {
        print("FaceCapture licenseKey invalid: \(error)")
        return false
    }
    return true
}
```

Objective C:

```
- (BOOL)application:(UIApplication *)application
        didFinishLaunchingWithOptions:(NSDictionary *)launchOptions {


        NSError* error = [FaceCapture.shared setLicenseKey:@"1234-5678-9ABC-DEFG"];

        if (error) {
          NSLog(@"FaceCapture licenseKey invalid: %@", error);
          return NO;
        }

        return YES;
}
```

To receive the IlluminateAI color matching results, when the process is complete, you will need to set up a target object, usually the calling **UIViewController**, as a delegate object.

Swift:

```
override func viewDidLoad() {
    super.viewDidLoad()

    if let error = FaceCapture.shared().setLicenseKey("1234-5678-9ABC-DEFG") {
       print("FaceCapture licenseKey invalid: \(error)")
       return
    }
    FaceCapture.shared().delegate = self
}
```

Objective C:

```
- (void)viewDidLoad {
    [super viewDidLoad];

    NSError* error = [FaceCapture.shared setLicenseKey:@"1234-5678-9ABC-DEFG"];
    if (error) {
       NSLog(@"FaceCapture licenseKey invalid: %@", error);
       return;
    }
    FaceCapture.shared.delegate = self;
}
```

And then the delegate calls:

Swift:

```swift
extension ViewController : FaceCaptureDelegate {

    func processingResults(success: Bool, result: RingoResult) {
        if success {
        // display UX to show matching information in info parameter
        } else {
            if let error = result.error {
                print("failure processing error: \(result.error)")
            } else {
                print ("failure processing")
            }
        }
    }
}
```

Objective C:

```objc
@interface ExampleVC () <FaceCaptureDelegate>
@end
@implementation ExampleVC
- (void)processingResults:(BOOL)success result:(RingoResult*)result {
    if (success) {
    // display UX to show matching information in info parameter
    } else {
        NSObject* error = result.error;
        if (error)
            NSLog(@"failure processing %@", error);
        else
            NSLog(@"failure processing");
    }
}
@end
```

The structure of the **RINGORESULT** is a parent structure that contains the finished calculations, and the matches for each product supported. Please note that it has changed slightly since the previous version.
It has been simplified into primarily arrays.

```
typedef struct {
      UInt8 r,g,b;
} RGBColor;

@interface RingoShade : NSObject
@property (nonatomic, strong, nonnull)  NSString*     shade;
@property (nonatomic, strong, nonnull)  NSString*     shadeId;
@property (nonatomic, strong, nonnull)  NSString*     shadeDesc;
@property (nonatomic)                   RGBColor      color;
@property (nonatomic)                   NSInteger     rank;
@property (nonatomic)                   BOOL          grid;
@end
typedef NSArray<RingoShade*>*  RingoShades;

// key:value Product Name: RingoShade[]
@interface RingoMatch: NSObject
@property (nonatomic, strong, nonnull)  NSString*     family;
@property (nonatomic, strong, nonnull)  NSString*     familyId;
@property (nonatomic, strong, nonnull)  NSString*     familyDesc;
@property (nonatomic, strong, nonnull)  RingoShades   shades;
@end
typedef NSArray<RingoMatch*>*  RingoMatches;

@interface RingoResult : NSObject
@property (nonatomic, strong, nonnull)  NSString*           userID;
@property (nonatomic, strong, nonnull)  RingoMatches        matches;
@property (nonatomic, strong, nonnull)  NSString*           defaultFamilyId;
@property (nonatomic, strong, nonnull)  NSString*           sessionId;
@property (nonatomic)                   NSUInteger          timestamp;
@property (nonatomic)                   float               L;
@property (nonatomic)                   float               minL, maxL;
@property (nonatomic)                   float               H;
@property (nonatomic)                   float               minH, maxH;
@property (nonatomic, strong, nonnull)  NSError*            error;
@property (nonatomic, strong, nonnull)  NSString*           errorMessage;
@property (nonatomic, strong, nonnull)  NSDictionary*       metadata;
@property (nonatomic, strong, nonnull)  id                  devInfo;
@end
```

The `userID` is the same as passed in originally via the `faceCapture.metadata()` call.

The `sessionId` and `timestamp` are useful for analytics – The `sessionId` is a unique hash that can be used as a reference key.

The `matches` field is an array of `RingoMatch` objects which represents each product family. By iterating on that array, and matching the `familyId` to the `RingoResult.defaultFamilyId,` that will be the desired product. The `shades` field are the matches for that family of products, as before, and consist of the `RingoShade` object. The main concern is to go off of the IDs rather than the names.

The `RingoShade` object are the actual “matches” per product.

`shade` *name of the product matched*
`shadeId` *id of the product suitable for URL linking*
`shadeDesc` *provided description of the product shown in the Results page*
`color` *rgb color to display – red, green, blue 0-255*
`rank` *integer -1,0, or 1, suggested match = 0, alternate match #1 = -1, alternate match #2 = 1*

By iterating on `shades`, and extracting the `color` rgb values, it is easy to construct a UX displaying the matches.

See XCode **SAMPLE** app project as a rudimentary, but fully functioning example of IlluminateAI SDK Integration.